

UDC: 004.896

Иерархическая когнитивная архитектура управления для мобильной робототехнической платформы

Аннотация

Предложена реализация трёхуровневой иерархической когнитивной архитектуры управления мобильным агентом с дифференциальным приводом в среде с препятствиями и переменными свойствами поверхности, объединяющая нижний уровень (TD3), средний (LQR) и верхний (обход по карте). Покомпонентный анализ показывает, что верхний уровень разгружает нижний при известной геометрии препятствий, но не компенсирует физических ограничений нижнего уровня при слабом сцеплении колёс.

Ключевые слова: обучение с подкреплением, TD3, LQR, иерархическое управление, мобильная робототехника, STRL, residual policy, когнитивный агент

1. Введение

Автономная навигация мобильного агента в среде с препятствиями и переменными свойствами поверхности объединяет обучение с подкреплением, классическое управление и иерархическое планирование. Один из устоявшихся подходов — трёхуровневая иерархическая когнитивная архитектура [1, 2, 3] с распределением поведения по горизонтам планирования. В диссертации [2] такая архитектура реализована для платформы МП-РМ «Зарница» с Q -обучением на нижнем уровне; в [4] направление развивается мультимодальной модельной компонентой.

В настоящей работе предложена реализация трёхуровневой архитектуры для *непрерывного* управления: нижний уровень — стратегия, обученная методом TD3 [5]; средний — LQR-регулятор; верхний — модуль обхода по известной карте. Архитектура протестирована в PyBullet на платформе Husky.

Вклад работы. (1) Реализация трёхуровневой иерархической архитектуры для непрерывного управления агентом. (2) Покомпонентный анализ распределения функций между уровнями на 65 конфигурациях ($7 \times 5 + 6 \times 5$) с $n = 10$ запусками на ячейку. (3) Открытый репозиторий с воспроизводимой реализацией.

2. Обзор работ

В задачах непрерывного управления применяются методы актора-критика [6, 5, 7, 8]. В настоящей работе используется TD3 [5] как алгоритм с устойчивым обучением на робототехнических задачах и эталонной реализацией в `stable-baselines3` [9].

Комбинирование обученной стратегии с классическим регулятором [10, 11] компенсирует ограничения каждого подхода в отдельности: обучение требует большого числа взаимодействий со средой, а классические контроллеры теряют точность при трении и неучтённой нелинейности. Линейно-квадратичный регулятор применяется к кинематике дифференциального привода через линеаризацию модели в координатах ошибки курса и поперечного отклонения.

Трёхуровневая иерархическая архитектура управления (STRL [1, 2, 3]) исторически развивается в поведенческой робототехнике и расширена в [4] мультимодальной модельной компонентой. Параллельное направление гибридизации обучения с подкреплением и методов оптимального управления систематизировано в обзорах [15, 16], с близкими подходами в [12, 13, 17]. Настоящая работа развивает указанное направление в части непрерывного управления и количественного анализа роли каждого уровня архитектуры.

3. Метод

3.1. Постановка задачи. Задача формализуется как марковский процесс принятия решений $\langle S, A, T, R, \gamma \rangle$ с непрерывными пространствами на плоской арене 10×10 м с препятствиями известной геометрии. Среда моделируется в PyBullet [14] на платформе Husky. Состояние агента

$$s_t = (\mathbf{r}_t, \theta_t, \mathbf{v}_t, \omega_t, \mathbf{g}_t, \ell_t)^\top \in \mathbb{R}^{n_s}, \quad (1)$$

включает координаты $\mathbf{r}_t = (x_t, y_t)$, ориентацию θ_t , линейную и угловую скорости $\mathbf{v}_t, \omega_t$, относительное положение цели $\mathbf{g}_t = \mathbf{r}_g - \mathbf{r}_t$, и лидарные измерения $\ell_t \in \mathbb{R}^{16}$ (только при наличии препятствий). Действие $a_t \in [-1, 1]^2$ — нормированные скорости левой и правой пар колёс, преобразуемые в физические $\omega_w \in [-10, 10]$ рад/с. Шаги физики и управления $1/240$ и $1/20$ с, $\gamma = 0.99$. Цель на расстоянии 2–5 м, радиус достижения 0.4 м. Эпизод завершается при достижении цели, перевороте, выходе за арену, столкновении или таймауте 500 шагов. Вознаграждение плотное (штраф за расстояние, шаговый штраф, терминальные бонус и штрафы).

3.2. Иерархическая архитектура управления. Архитектура, восходящая к трёхуровневой схеме [1] и развитая для когнитивных агентов в [2, 3] адаптирована для непрерывного управления и включает три уровня.

Нижний уровень — обученная методом TD3 стратегия $\pi_{\text{TD3}}(s) \rightarrow a$, опирающаяся на текущее наблюдение.

Средний уровень — LQR-регулятор по линеаризованной модели поперечного отклонения и отклонения курса агента от прямой к цели. Регулятор выдаёт скалярную коррекцию $u_{\text{LQR}} \in [-u_{\text{max}}, u_{\text{max}}]$ к угловой скорости, комбинируемую с действием нижнего уровня по схеме остаточной коррекции [10, 11].

Верхний уровень — модуль обхода препятствий по известной карте (MapAware) с индикатором активации $\mathbb{K}_{\text{map}}(s) \in \{0, 1\}$. При приближении агента к препятствию в передней полусфере на расстояние ниже $d_{\text{takeover}} = 1.5$ м индикатор переключается в 1, при выходе за $d_{\text{release}} = 2.5$ м — в 0; гистерезис предотвращает дребезг. Пороги согласованы с кинематикой: $v_{\text{max}} = \omega_{\text{max}} \cdot r_w \approx 1.8$ м/с при $\omega_{\text{max}} = 10$ рад/с и $r_w = 0.178$ м, тогда $d_{\text{takeover}}/v_{\text{max}} \approx 0.83$ с обеспечивает запас для смены курса. В текущей реализации уровень замещает управление в опасных конфигурациях, не предлагая последовательности подделей; полноценный планировщик — естественное направление развития.

Конечное действие формируется как $a_t = a_{\text{MapAware}}(s_t)$ при $\mathbb{K}_{\text{map}}(s_t) = 1$, иначе как $\text{clip}(a_{\text{TD3}}(s_t) + \alpha[-1, 1]^T u_{\text{LQR}}(s_t), -1, 1)$, где $\mathbb{K}_{\text{map}}$ — индикатор активности верхнего уровня, $\alpha = 0.2$ — весовой коэффициент остаточной коррекции.

3.3. Нижний уровень: TD3. В качестве базовой стратегии используется TD3 [5], актор-критик с двойной оценкой функции ценности, задержанным обновлением актора и буфером переходов; функции ценности и стратегия — полносвязные сети со слоями 400 и 300 (ReLU). Гиперпараметры стандартные для TD3 [9]: минибатч 256, Adam с $\text{lr } 10^{-3}$, $\tau = 0.005$, $d = 2$, $\sigma_t = 0.2$, $c = 0.5$, шум $\mathcal{N}(0, 0.3)$, буфер $|\mathcal{D}| = 2 \cdot 10^5$.

Вектор наблюдения $o_t = (\cos \theta_t, \sin \theta_t, \mathbf{v}_t, \omega_t, \mathbf{g}_t, \ell_t)^T$ включает ориентацию, скорости и относительное положение цели $\mathbf{g}_t$. Лидарные измерения $\ell_t \in \mathbb{R}^{16}$ получаются агрегированием 360 лучей (паспортные характеристики LDS-01 TurtleBot 3 Burger [18]) в 16 секторов по 22.5° операцией поиска минимума $\ell_t^{(k)} = \min_{j \in \mathcal{B}_k} d_t^{(j)}$. Такая агрегация гарантирует обнаружение тонких препятствий, которые могут проваливаться между лучами в низкоразрешённом сканировании. Наблюдение привилегированное, что отделяет управление от восприятия.

3.4. Средний уровень: LQR-residual. LQR-регулятор работает с вектором ошибки $e = [e_y, e_\theta]^T$ (поперечное отклонение и отклонение курса от прямой к цели) и выдаёт коррекцию $\omega = -Ke$ с матрицей K , найденной из решения алгебраического уравнения Риккати для линеаризованной модели с $Q = \text{diag}(2.0, 1.0)$ и $R = 5.0$ (преобладание штрафа на управление обеспечивает мягкие коррекции).

В реализации e_y обнуляется на каждом шаге: опорная прямая переопределяется от текущей позы агента к цели, и закон вырождается в $\omega = -k_\theta e_\theta$, P-регулятор по углу с коэффициентом из LQR-задачи. Выход ограничивается $|\omega| \leq u_{\text{max}} = 0.3$.

3.5. Верхний уровень: обход с использованием карты. Уровень получает карту препятствий в виде списка $\{(o_i^x, o_i^y, r_i)\}$ и вычисляет расстояния до краёв

препятствий в передней полусфере $\pm\pi/2$ относительно курса агента. Активация определяется двупороговой логикой с гистерезисом: индикатор $\mathbb{I}_{\text{map}}$ переключается в 1 при пересечении d_{takeover} и обратно в 0 при пересечении d_{release} . При активном уровне управление задаётся прямолинейным движением со скоростью $v_{\text{sup}} = 0.4$ м/с и угловой скоростью $\omega_{\text{sup}} = k_{\text{turn}} \cdot \text{sign}(-\theta_{\text{nearest}})$, $k_{\text{turn}} = 1.5$, где θ_{nearest} — угол на ближайшее препятствие. Простота алгоритма преднамеренна: она позволяет изолировать модельное знание о среде. Для сопоставления модельного и сенсорного знания используется также альтернативный модуль обхода по лидарным дальностям (LidarAvoid).

4. Эксперименты

4.1. Конфигурация и метрики. Эксперименты проведены на сервере 8 ядер Ryzen 9 9950X / 16 ГБ RAM без GPU, PyBullet 3.2 в безголовом режиме, 6 параллельных сред через SubprocVecEnv, сиды $\{200, \dots, 209\}$, $n = 10$ запусков на ячейку. Основная метрика — доля успешных эпизодов (success rate). Дополнительно фиксируются длина эпизода, гладкость управления, максимальная линейная скорость и доля времени в верхнеуровневом режиме. Сравниваются пять архитектурных режимов: TD3, TD3+LQR, TD3+LidarAvoid, TD3+MapAvoid и TD3+MapAvoid+LQR.

4.2. Эксперимент А: множественные конфигурации препятствий. Семь детерминированных конфигураций препятствий: одиночное препятствие на прямой, два со смещением, коридор из трёх, барьер с проходом, слалом, диагональное расположение цели, широкое препятствие.

Сценарий	TD3	+LQR	+Lidar	+Map	+Map+LQR
single_on_line	0/10	0/10	10/10	10/10	10/10
two_offset	0/10	0/10	10/10	10/10	10/10
three_corridor	0/10	0/10	0/10	10/10	10/10
barrier_with_gap	10/10	10/10	10/10	0/10	0/10
slalom	0/10	0/10	0/10	0/10	0/10
diagonal_goal	10/10	10/10	10/10	10/10	10/10
wide_obstacle	0/10	0/10	0/10	10/10	10/10

Таблица 1. Эксперимент А: доля успешных эпизодов (из 10 запусков на сидах 200–209) для семи конфигураций препятствий и пяти архитектурных режимов. Все ячейки 100% детерминированы.

Сценарий *diagonal_goal* без препятствий на прямой решается всеми режимами и служит контролем. Сценарий *slalom* не решается ни одним режимом, фиксируя границу применимости архитектуры.

В сценариях *single_on_line* и *two_offset* оба варианта верхнего уровня обеспечивают полный успех, тогда как нижний и средний уровни в одиночку не справляются: обученная стратегия идёт по прямой к цели, а LQR-коррекция общую стратегию не меняет. В *three_corridor* полный успех достигает только модельный верхний уровень, что соответствует роли стратегического уровня при известной структуре среды. Обратный паттерн наблюдается в *barrier_with_gap*, где модельный уровень обходит барьер целиком, хотя геометрический проход проходим напрямую, и в *wide_obstacle*, где аппроксимация цилиндром малого радиуса не активирует MapAvoid на нужной дистанции. Эти сценарии указывают, что полезность каждого варианта верхнего уровня зависит от соответствия внутренней модели реальной геометрии среды.

Воспроизводимость. Результаты в таблице 1 полностью детерминированы: каждая ячейка имеет однозначный исход для всех сидов. Детерминированность — следствие фиксированности PyBullet, весов стратегии и отсутствия стохастичности в верхних уровнях.

4.3. Эксперимент В: переменные свойства поверхности. Реализованы три типа поверхности: *NORMAL* (трение 0.9), *ICE* (0.05) и *SAND* (0.6 с дополнительным сопротивлением качению), каждый в конфигурациях *empty* и *with_obstacle*, $n = 10$. В пустой арене нижний уровень успешен на всех поверхностях, автоматически адаптируя скорость. Принципиальный результат — сценарий ICE+obstacle, где полного успеха (10/10) достигает только полная трёхуровневая конфигурация TD3+MapAvoid+LQR: без среднего уровня низкое трение приводит к проскальзыванию при попытке обхода. Это эмпирически подтверждает взаимодополнение уровней — ни один в отдельности не достаточен при физических ограничениях нижнего уровня.

5. Обсуждение

Средний уровень не меняет стратегических решений нижнего уровня — включение LQR не меняет долю успеха ни в одной ячейке таблицы 1, но подавляет высокочастотные колебания курса. *Верхний уровень разгружает нижний при адекватной геометрической модели*: в *single_on_line* и *two_offset* верхний уровень обеспечивает полный успех; обратный эффект в *barrier_with_gap* и *wide_obstacle* указывает на чувствительность к точности представления геометрии. *Модельное знание не компенсирует физических ограничений*: на ICE+obstacle полный успех достигается только полной трёхуровневой архитектурой. *Ограничения*: простейший геометрический алгоритм без оптимизационного планирования, привилегированное наблюдение нижнего уровня, отсутствие реальной платформы.

6. Заключение

Реализована и эмпирически исследована трёхуровневая иерархическая архитектура управления мобильным агентом с дифференциальным приводом [2, 3]: нижний уровень — стратегия TD3, средний — LQR, верхний — модуль обхода по карте. Покомпонентный анализ на семи конфигурациях препятствий и трёх типах поверхности показывает, что верхний уровень разгружает нижний при известной геометрии, но не компенсирует слабое сцепление колёс; средний улучшает гладкость управления. Направления развития: верхний уровень с оптимизационным планировщиком, интеграция с мультимодальной компонентой [4], перенос на реальную платформу с ROS.

ЛИТЕРАТУРА

1. Gat E. On three-layer architectures // *Artificial Intelligence and Mobile Robots* / D. Kortenkamp, R. Bonasso, R. Murphy (eds.). AAAI Press, 1998. P. 195–210.
2. Киселёв Г. А. Интеллектуальная система планирования поведения коалиции робототехнических агентов со STRL архитектурой: дис. . . канд. техн. наук. Москва, 2020. 117 с.
3. Киселёв Г. А., Панов А. И. Иерархическое психологически-инспирированное планирование для интеллектуальных динамических систем // *Известия РАН. Теория и системы управления*. 2020. № 1. С. 123–137.
4. Вейценфельд Д. А., Киселёв Г. А. Метод синтеза поведения когнитивного агента на основе обработки мультимодальных сигналов // *Моделирование и анализ данных*. 2024. Т. 14. № 4. С. 45–62.
5. Fujimoto S., van Hoof H., Meger D. Addressing function approximation error in actor-critic methods // *Proc. of ICML*. 2018. P. 1587–1596.
6. Lillicrap T., Hunt J., Pritzel A. et al. Continuous control with deep reinforcement learning // *Proc. of ICLR*. 2016.
7. Haarnoja T., Zhou A., Abbeel P., Levine S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor // *Proc. of ICML*. 2018. P. 1861–1870.
8. Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal policy optimization algorithms // *arXiv preprint arXiv:1707.06347*. 2017.
9. Raffin A., Hill A., Gleave A. et al. Stable-Baselines3: Reliable reinforcement learning implementations // *Journal of Machine Learning Research*. 2021. V. 22. № 268. P. 1–8.
10. Silver T., Allen K., Tenenbaum J., Kaelbling L. Residual policy learning // *arXiv preprint arXiv:1812.06298*. 2018.
11. Johannink T., Bahl S., Nair A. et al. Residual reinforcement learning for robot control // *Proc. of ICRA*. 2019. P. 6023–6029.
12. Levine S., Finn C., Darrell T., Abbeel P. End-to-end training of deep visuomotor policies // *Journal of Machine Learning Research*. 2016. V. 17. P. 1–40.
13. Cheng R., Orosz G., Murray R. M., Burdick J. W. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks // *Proc. of AAAI*. 2019. P. 3387–3395.
14. Coumans E., Bai Y. PyBullet, a Python module for physics simulation for games, robotics and machine learning. 2016–2021. URL: <http://pybullet.org>
15. Hewing L., Wabersich K. P., Menner M., Zeilinger M. N. Learning-Based Model Predictive Control: Toward Safe Learning in Control // *Annual Review of Control, Robotics, and Autonomous Systems*. 2020. V. 3. P. 269–296.
16. Brunke L., Greeff M., Hall A. W., Yuan Z., Zhou S., Panerati J., Schoellig A. P. Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning // *Annual Review of Control, Robotics, and Autonomous Systems*. 2022. V. 5. P. 411–444.
17. Kahn G., Villafior A., Ding B., Abbeel P., Levine S. Self-supervised deep reinforcement learning with generalized computation graphs for robot navigation // *Proc. of IEEE ICRA*. 2018. P. 5129–5136.
18. Robotis Co., Ltd. LDS-01 (360 Laser Distance Sensor): Spec sheet // URL: https://emmanual.robotis.com/docs/en/platform/turtlebot3/appendix_lds_01/. 2017.